

مشروع مادة بنى المعطيات نظام الاستعلام الفوري في "صيدلية الوادي"

البرنامج الدراسي:

دبلومة تقانة المعلومات

بنية البيانات المستخدمة:

جدول التقطيع — Hash Table

معالجة التصادم:

السلاسل المتشعبة — Chaining

اسم الطالبة:

مروة الصحني

أستاذ المادة:

م. زكريا صافي

جدول المحتويات | Table of Contents

طرح المشكلة	
بنية البيانات المقترحة - Hash Table	
دالة التقطيع - Hash Function	
معالجة التصادم - Chaining	
شرح الدوال المستخدمة	
الكود المصدري الكامل	
مثال على تشغيل البرنامج	

طرح المشكلة | Problem Statement

تتعامل صيدلية "الوادي" المركزية مع مئات الآلاف من الأصناف الدوائية، ويمتلك كل دواء رقم باركود فريد مكون من 4 أرقام. تعاني الصيدلية حالياً من بطء شديد في عملية البيع؛ لأن النظام الحالي يبحث عن الدواء عبر فحص السجلات واحداً تلو الآخر (البحث الخطي)، مما يجعل وقت الانتظار يزداد كلما زاد عدد الأدوية المخزنة.

بنية البيانات المقترحة | Proposed Data Structure

الحل الأمثل هو استخدام جدول التقطيع (Hash Table). هذه البنية تسمح بالوصول المباشر لأي سجل في خطوة زمنية واحدة وثابتة ($O(1)$)، بغض النظر عن حجم البيانات سواء كان في النظام 100 دواء أو مليون دواء. يتم استخلاص موقع التخزين من رقم الباركود نفسه عبر دالة التقطيع.

مقارنة: البحث الخطي مقابل جدول التقطيع

المعيار	البحث الخطي (المشكلة)	جدول التقطيع (الحل)
زمن البحث	يزيد مع البيانات - $O(n)$	ثابت دائماً - $O(1)$
طريقة الوصول	فحص واحد تلو الآخر	مباشر عبر الباركود
مع نمو البيانات	يتباطأ مع الزيادة	لا يتأثر
الأداء في الصيدلية	بطيء جداً	ممتاز

دالة التقطيع — Hash Function

تستخدم طريقة باقي القسمة (Division Method): $\text{index} = \text{barcode} \% \text{TABLE_SIZE}$ حجم الجدول = 1000، وهذا يعني أن باركود 1573 يخزن في الخلية 573، وباركود 3001 يخزن في الخلية 1، وهكذا.

$$\text{index} = \text{barcode} \% 1000$$

أمثلة على دالة التقطيع:

الباركود	العملية	الموقع (index)
1573	$1573 \% 1000$	573
2573	$2573 \% 1000$	573 ← 1573
3001	$3001 \% 1000$	1
5000	$5000 \% 1000$	0

معالجة التصادم — Chaining

التصادم يحدث عندما يعطي باركودان مختلفان نفس موقع التخزين. الحل المستخدم هو السلاسل المتشعبة (Chaining): كل خلية في الجدول تحتوي على سلسلة مرتبطة (Linked List) تخزن جميع الأدوية التي تشاركت نفس الموقع. هذا يضمن عدم ضياع أي سجل دوائي.

شرح الدوال المستخدمة | Function Descriptions

`hashFunction(barcode)`

دالة التقطيع

تحتسب موقع التخزين: $\text{barcode} \% 1000$. تعمل بـ $O(1)$ دائما.

`insert(barcode, name, price)`

الإدراج

تضيف دواءا جديدا عبر حساب موقعه وإدراجه في رأس سلسلة الموقع.

`retrieve(barcode)`

البحث اللحظي

تنتقل مباشرة للموقع ثم تبحث في سلسلته للحصول على الاسم والسعر.

`demonstrateCollision()`

معالجة التصادم

توضح عمليا كيف يتعامل النظام مع باركودين يعطيان نفس الموقع.

`update(barcode, newPrice)`

التحديث

تنتقل مباشرة للموقع وتعديل سعر الدواء دون المرور على بقية العناصر.

الكود المصدري الكامل | Full Source Code

MarwaAlSahni.cpp

```

#include <iostream>
#include <string>
using namespace std;

const int TABLE_SIZE = 1000;

struct Node {
    int barcode;
    string name;
    double price;
    Node* next;
    Node(int b, string n, double p) : barcode(b), name(n), price(p), next(nullptr) {}
};

Node* table[TABLE_SIZE];

int hashFunction(int barcode) {
    return barcode % TABLE_SIZE;
}

void insert(int barcode, string name, double price) {
    int index = hashFunction(barcode);
    Node* cur = table[index];
    while (cur != nullptr) {
        if (cur->barcode == barcode) {
            cout << "  [" << barcode << "  ." << endl;
            return;
        }
        cur = cur->next;
    }
    Node* newNode = new Node(barcode, name, price);
    newNode->next = table[index];
    table[index] = newNode;
    cout << "  [" << name << "  | : " << index << endl;
}

void retrieve(int barcode) {
    int index = hashFunction(barcode);
    Node* cur = table[index];
    while (cur != nullptr) {
        if (cur->barcode == barcode) {
            cout << "  [" << cur->name << endl;
            cout << "      : " << cur->price << "  ." << endl;
            return;
        }
        cur = cur->next;
    }
    cout << "  [" << barcode << endl;
}

void demonstrateCollision() {
    int b1 = 1573, b2 = 2573;
    cout << "  " << b1 << " => : " << hashFunction(b1) << endl;
    cout << "  " << b2 << " => : " << hashFunction(b2) << endl;
    cout << "  => -      ." << endl;
}

```

```

insert(b1, "Paracetamol 500mg", 150);
insert(b2, "Aspirin 100mg", 200);
cout << "    =>    ." << endl;
}

void update(int barcode, double newPrice) {
    int index = hashFunction(barcode);
    Node* cur = table[index];
    while (cur != nullptr) {
        if (cur->barcode == barcode) {
            double oldPrice = cur->price;
            cur->price = newPrice;
            cout << "    [] : " << cur->name << endl;
            cout << "        : " << oldPrice << " ." << endl;
            cout << "        : " << newPrice << " ." << endl;
            return;
        }
        cur = cur->next;
    }
    cout << "    []    " << barcode << endl;
}

int main() {
    for (int i = 0; i < TABLE_SIZE; i++) table[i] = nullptr;

    cout << "===== " << endl;
    cout << "    -    " << endl;
    cout << "===== " << endl;

    int choice = 0;
    while (choice != 5) {
        cout << "\n---    ---" << endl;
        cout << "1.            (Insertion)" << endl;
        cout << "2.            (Retrieval)" << endl;
        cout << "3.            (Collision Handling)" << endl;
        cout << "4.            (Update)" << endl;
        cout << "5. " << endl;
        cout << " : ";
        cin >> choice;
        cin.ignore();

        switch (choice) {
            case 1: {
                int bc;
                string nm;
                double pr;
                cout << "    (4) : ";
                cin >> bc;
                cin.ignore();
                cout << " : ";
                getline(cin, nm);
                cout << " (.): ";
                cin >> pr;
                cin.ignore();
                insert(bc, nm, pr);
                break;
            }
            case 2: {
                int bc;
                cout << " : ";
                cin >> bc;
            }
        }
    }
}

```



```

        cin.ignore();
        retrieve(bc);
        break;
    }
    case 3:
        demonstrateCollision();
        break;
    case 4: {
        int bc;
        double np;
        cout << " : ";
        cin >> bc;
        cin.ignore();
        cout << " (.): ";
        cin >> np;
        cin.ignore();
        update(bc, np);
        break;
    }
    case 5:
        cout << "\n !" << endl;
        break;
    default:
        cout << " [!] ." << endl;
    }
}

return 0;
}

```

مثال على تشغيل البرنامج | Sample Program Run

فيما يلي مثال كامل يوضح تشغيل البرنامج وإخراجه بعد تنفيذ جميع العمليات الأربع:

```
=====
-
=====

--- ---
: 1
(4 ): 1573
: Vitamin C
(.): 500
[+] : Vitamin C | : 573

--- ---
: 1
(4 ): 3001
: Panadol Extra
(.): 350
[+] : Panadol Extra | : 1

--- ---
: 2
: 1573
[] : Vitamin C
: 500 .

--- ---
: 3
1573 => : 573
2573 => : 573
=> - .
[!] 1573 .
[+] : Aspirin 100mg | : 573
=> .

--- ---
: 4
: 1573
(.): 600
[] : Vitamin C
: 500 | : 600 .

--- ---
: 5
!
```

تعليمات التشغيل | How to Run

لتجميع البرنامج وتشغيله، استخدم الأوامر التالية في terminal:

```
g++ -o MarwaAlSahni MarwaAlSahni.cpp -std=c++11
```

```
./MarwaAlSahni (Linux/Mac) | MarwaAlSahni.exe (Windows)
```